

Assignment 6: Graphical User Interface

Gabriel Castillo (gac232), Leo Qian (rq48), Wallace Wei (wzw4), Katherine Hu (kyh29)

Summary

In A6, we were tasked with two main tasks: Dijkstra's algorithm / BinaryHeap and the GUI. We first tackled the GUI design challenge. The most challenging part of this assignment would have been implementing the UI, especially given the lack of experience any team members had working with JavaFX. Luckily, one of our members had experience with front end development in iOS applications which helped them quickly learn SceneBuilder.

A notable aspect of our implementation is our completely scalable view. The user can zoom in as much as they want the world hex view will always stay super crisp. This also made our program run faster since we were only rendering the part of the world the viewer can see.

Specification Changes

We have kept our use of concurrent threads to 2, one for the view and one for the controller and model. This simplifies the code and reduces the chances of producing bugs from parallel threads.

Another specification change is that our world is rendered upside down, with the coordinate (0,0) being at the top left. This is different from the images shown in the specs that usually have it at the bottom left. Also while the simulation is running the user is not allowed to load in a random critter or select a hex to load a critter onto, they are allowed to load a new world or a randomly generated world, but this will pause the current simulation and override it.

Design and Implementation

Architecture

In this program we have used the Model View Controller design pattern. The model in our program is the data structures we created and implemented in A4 including World, Tiles, Critter, and other classes used to maintain the state of the world. This model will be managed by the controller, as explained later.

For the view aspect, we created a View class responsible for handling everything related to the graphical user interface. Specifically it launches the application loading the FXML file we created using SceneBuilder, it handles any user interaction including clicks, scrolls, and resizing by appropriately changing the view in response to these events, i.e. clicking any buttons or resizing the world. Most importantly, it draws the world out on a canvas. To render the hexes we

used polygon objects that store all six coordinates of a given hex and used it for rendering (by drawing over these coordinates) and mouse click events.

Finally, the Controller class is responsible for running and managing the simulation, or model (in MVC terms). In order to do this effectively, it is run in the same thread as the model, the view however is run on a different thread in order to keep the GUI response while running tasks in the background. In order for the View to communicate any background tasks that the user calls on the GUI with the Controller we use a TaskQueue. The controller checks the TaskQueue after every simulation step (or in a fixed interval if no world is loaded) and attempts to execute the tasks within the queue. For example when a “load critter from file” task is added to the TaskQueue, it will be ready for the Controller to run after a simulation step. Additionally, the view has some getter methods that return primitive values (so as to prevent giving other classes pointers to private variables), and it has some methods that update the view which can be called from the Controller class.

These are all of our classes, and thus our Module Dependency Diagram will look identical to the MVC design's diagram (except replacing the “Model” class with our Simulation class), so we think it's not necessary here.

We based our GUI off of the following wireframe (left), but specifically it is split into three areas: the information tab on the left, the menu bar on top and the world view on the right. The menu bar allows users to easily access buttons. The simulation is displayed by the world view, presenting some critters, some food and several rock hexes. The inspection tab actively switches between critter as well as world info, with the critter tab activated when a certain critter is clicked. Buttons control simulation speed. They also control stepping at the bottom of the tab. This layout importantly improves screen efficiency, allowing multiple users to maintain a visible world view when they resize the window, in addition to keeping many necessary buttons within reach.

To explain the GUI workflow of a user, first they would either load a world or generate a new world (which is completely random) from the menu. If they would like to load a random number of critters they can click on the Critter button on the menu, and they can right click a given hex and then click the “Load Critter” button. They can select the speed they want the world to go at and click “Play” to play the world at the speed that is currently selected. If they would like to go a single step at a time they can click the “Step” button. The “Critter” tab on the left hand side below the menu will be disabled until a critter is selected on the world. The user can click and drag to move around the world and scroll to zoom in and out. If the user drags too far away from the center of the world, they can click the bulls-eye icon on the top right corner to re-center their view of the world.

Code Design

We will use a HashMap for the BinaryHeap required to implement Dijkstra's shortest path algorithm in order for amortized $O(1)$ lookup time. For managing the TaskQueue we used a LinkedList, again for $O(1)$ operation time.

An interesting Code Design choice was to redraw the canvas. At first, we drew out the canvas one time while it was loaded and then we zoom in on and translate that large canvas as the user scrolls and pans the view. This resulted in a significantly low quality view when the user zooms in for worlds that are more than 200 by 200 hexes large. In order to solve this problem, we decided to redraw the view in response to any user action rather than manipulating the larger view. This made our FPS faster than the previous version because now the program does not need to render things the user does not see (through a bounding box).

To implement Dijkstra's shortest path algorithm, we defined the graph as an undirected graph with 6 nodes for each hex tile and each edge having a weight of 1. This allows us to have 6 nodes corresponding to each unique direction for each tile, which allows us to run Dijkstra's shortest algorithm on the graph.

If Dijkstra's algorithm ever reaches one of 6 nodes corresponding to a food tile, we first check if the last edge was a backward edge (critter path moves backward), if not, we attempt to save that shortest distance to a minimum value. This allows us to find the shortest path from any tile to any food tile, however the result of ping requires us to end on facing the critter so we will subtract 30 from the final ping result to undo the last step of moving onto the food tile.

Each node on every path is represented by a Priority object that records the move, dist, and energyCost. The move value records the first move direction of the critter and is unchanging after declaration. The dist records every time the path moves forward and terminates the Dijkstra's path after 15 nodes, and the energyCost records the total energy cost used in the return value of Ping.

Programming

We did Top-Down Implementation for this assignment. We first considered the relationship between the classes rather than implementing unlike A4, since we will only be implementing a handful of classes, and the benefits of keeping good design helps prevent inefficiency and allows us to utilize threads to its fullest ability. In fact, it is much easier to test the whole thing, even missing some functionality like Controller, just testing to see the GUI. We did top-down because the MVC Controller class will be the last to implement, which allows the entire group to take part in the implementation of the MVC Controller class and makes it easier for debug and testing.

Testing

We tested the BinaryHeap, GUI, and Ping for this project. For the BinaryHeap, we explicitly tested the running time, and plotted graphs of its runtime as the input scaled (up to 1 million elements) to make sure that the operations (like increasePriority, insert, peak, and poll) are $O(\log n)$ time complexity, and not acting abnormally. This is much like our testing of the hash tables in A3. You can find these graphs in the performance folder in the test folder.

We got in other people to test the GUI, and they let another group to play around with the GUI, resizing windows, pressing all the buttons. They allow for feedback on our design, but also how responsive our GUI is, and whether they can discover any new button combinations that can softlock it. They remarked that the GUI seemed fine.

We also tested the GUI by loading in all the world test cases in a5, as well as all the critter files we made in a5, and they passed.

We also manually tested the functions of BinaryHeap to make sure that it worked, as well as writing out a few manual tests for the new *Ping* critter sense, similar to how we tested the *smell*, *nearby*, and *ahead* critter sense in A5. These tests seem to have passed, as BinaryHeap's functions worked and satisfied the class invariants, as well as Poll and Peek returning the highest priority elements, increasePriority updating the priority and letting it go first if it has a high enough priority, and that insertion works on unique items and that size increases every time an item is inserted.

We tested Ping to work on the manual tests as seen in figures 5 and 6 in the Project Spec, and it gave the same result.

We also tested some of our custom critter programs, like prolific and forager, and to the naked eye, forager seemed to be fine, but when we tested prolific at first, it wasn't working, because the critters immediately died, and the fix happened when we forgot an "and" in our critter language, and turned a = to !=.

Work Plan

(Leo): Fix problems identified in A4 and A5.

(Wallace & Gabriel): Implement a GUI to display the state of the critter world.

(Everyone): Connect the simulation engine from Assignment 5 to the display. This means allowing the display to update as the simulation progresses and to start, stop, and step the simulation in a performant manner. Also implement the loading of critter and world files and the placement of critters into an existing world. (Implement **Controller** from MVC)

(Wallace & Gabriel): Make the GUI front end highly responsive to user input.

(Leo & Katherine): Implement the DynamicPriorityQueue interface.

(Leo & Katherine): Extend your tokenizer, parser, and AST to support ping, then implement ping specified above using Dijkstra's shortest-path algorithm so that critters have a new sense to find food.

(Everyone): Solve the written problems.

(Leo & Katherine): Test run-time complexity and functionality of BinaryHeap methods.

(Leo & Katherine): Test *Ping* Critter Sense

(Gabriel & Wallace): Test View and Controller according to Test plan.

Known Problems

none

Comments

none